

Scalable Market Basket Analysis & Cross-Selling Intelligence System

Project Team / Authors:	Tanaya Salunke & Sankhya Londhe
Academic Context:	Final Year Honors Project Proposal Department of Computer Science
Domain & Field:	Data Science, Data Mining, Machine Learning, & Retail Business Analytics
Benchmark Dataset:	Online Retail II Dataset (1,067,371 Raw Records, 2009–2011)
Proposal Date:	September 2026

1. Executive Summary

In contemporary retail and e-commerce environments, massive volumes of transaction logs are generated daily. Uncovering implicit associations between products purchased together provides critical commercial leverage for optimizing store layouts, organizing digital cross-selling prompts, designing promotional bundles, and managing inventory replenishment.

This project proposes a scalable, end-to-end **Market Basket Analysis (MBA)** and Product Cross-Selling Intelligence platform authored by **Tanaya Salunke** and **Sankhya Londhe**. Operating on the benchmark **Online Retail II** dataset (containing over 1.06 million transaction rows across 2009–2011), the system ingests raw transaction data, performs rigorous data hygiene, encodes transactions using memory-efficient sparse boolean matrices, benchmarks frequent pattern mining algorithms (**Apriori vs. FP-Growth**), extracts statistical association rules, filters out misleading spurious correlations, and translates mined patterns into actionable retail business strategies.

Furthermore, the system delivers an interactive **Streamlit Web Application** for real-time rule exploration and product cross-sell recommendations, alongside an **out-of-core batch processing architecture** and **PySpark FP-Growth reference integration** for multi-terabyte dataset scaling.

2. Problem Statement & Motivation

2.1 Problem Statement

Retail businesses collect vast amounts of point-of-sale (POS) and online transaction data, yet frequently fail to extract actionable product co-occurrence patterns. Key operational challenges include:

- Product Placement Inefficiencies:** Inability to identify which products drive impulse co-purchases, leading to sub-optimal aisle and shelf layouts.
- Ineffective Cross-Selling:** E-commerce platforms display generic or static product recommendations rather than data-driven 'Frequently Bought Together' pairs.
- Memory Bottlenecks in Association Mining:** Standard dense multi-hot encoding of high-cardinality retail catalogs leads to severe memory exhaustion ($O(N \times M)$ RAM consumption).
- Algorithmic Inefficiencies at Scale:** Traditional algorithms like Apriori suffer from combinatorial candidate itemset generation (C_k) and repeated database scans when candidate support thresholds are lowered.
- Spurious / Misleading Rules:** Naive association rule mining often surfaces rules with high confidence but low lift ($Lift \approx 1.0$), misleading retailers into bundling products that customers purchase independently anyway.

2.2 Motivation

By building a scalable, automated pipeline that addresses memory bottlenecks, prunes misleading rules, benchmarks algorithm efficiency, and presents insights visually via a web application, this project bridges the gap between complex pattern mining theory and real-world retail decision-making.

3. Project Objectives & Scope

3.1 Primary Objectives

- **Data Ingestion & Hygiene Engine:** Build an automated cleaner to filter returns/cancellations (InvoiceNo starting with 'C'), zero/negative quantities and prices, and non-product service charges (POSTAGE, MANUAL, BANK CHARGES).
- **Memory-Efficient Encoding:** Group transactions into invoice baskets and construct SciPy sparse boolean matrices (csr_matrix), achieving >95% matrix sparsity and >90% RAM footprint reduction.
- **Algorithmic Benchmarking:** Implement and compare Apriori (mlxtend.apriori) and FP-Growth (mlxtend.fpgrowth) across multiple minimum support thresholds ($\text{min_support} \in \{0.015, 0.02, 0.03, 0.05\}$) in terms of runtime execution (seconds), peak RAM footprint (MB), and itemset yield.
- **Rule Mining & Misleading Rule Pruning:** Compute Support, Confidence, Lift, Leverage, and Conviction for all candidate rules. Detect and flag misleading rules ($\text{Lift} < 1.2$).
- **Business Insight & Network Graph Layer:** Build NetworkX product co-occurrence graphs, execute regional segment analysis (UK, Germany, France, EIRE), and formulate concrete retail action plans.
- **Out-of-Core Scalability & PySpark Architecture:** Develop a chunked batch processor (50,000-row chunks) and map distributed FP-Growth execution to Apache Spark (pyspark.ml.fpm.FPGrowth).
- **Interactive Web Application:** Deploy a Streamlit dashboard featuring parameter sliders, sortable rule tables, CSV export, network visualizer, and a product cross-sell recommendation lookup tool.
- **Testing & Quality Assurance:** Implement unit tests using pytest to ensure 100% verification across all pipeline stages.

3.2 Scope Boundary

- **In-Scope:** Transaction-level market basket association mining, sparse matrix encoding, Apriori vs. FP-Growth benchmarking, rule filtering, network graph generation, regional analysis, Streamlit app, out-of-core batching, and PySpark reference module.
- **Out-of-Scope:** Real-time stream processing (e.g., Apache Kafka live event streaming) and customer RFM demographic modeling.

4. Theoretical Background & Mathematical Methodology

4.1 Association Rule Metrics

Given a transaction set T and itemsets A (antecedent) and C (consequent):

1. **Support (S):** Measures rule frequency across all transactions:

$$\text{Support}(A \rightarrow C) = P(A \cap C) = \text{Count}(A \cup C) / |T|$$
2. **Confidence (C):** Measures conditional probability of purchasing C given A :

$$\text{Confidence}(A \rightarrow C) = P(C | A) = \text{Support}(A \cup C) / \text{Support}(A)$$
3. **Lift (L):** Measures how much more often A and C occur together than expected if independent:

$$\text{Lift}(A \rightarrow C) = P(A \cap C) / (P(A) \cdot P(C)) = \text{Confidence}(A \rightarrow C) / \text{Support}(C)$$
 - $\text{Lift} > 1.0$: Positive co-occurrence association.
 - $\text{Lift} = 1.0$: Statistical independence (spurious association).
 - $\text{Lift} < 1.0$: Negative correlation.
4. **Leverage:** Difference between observed joint support and expected independent support:

$$\text{Leverage}(A \rightarrow C) = \text{Support}(A \cup C) - \text{Support}(A) \cdot \text{Support}(C)$$

5. **Conviction:** Ratio of expected frequency of incorrect prediction to observed incorrect prediction:
$$\text{Conviction}(A \rightarrow C) = (1 - \text{Support}(C)) / (1 - \text{Confidence}(A \rightarrow C))$$

4.2 Algorithm Comparison: Apriori vs. FP-Growth

- **Apriori Algorithm:** Uses level-wise candidate generation (C_k) and candidate pruning based on the Apriori property (all non-empty subsets of a frequent itemset must also be frequent). Requires k database passes for itemsets of length k.
- **FP-Growth Algorithm:** Uses a compact FP-Tree (Frequent Pattern Tree) trie structure. Encodes transactions in 2 database passes and mines frequent patterns by constructing conditional FP-trees without candidate generation.

5. Proposed System Architecture & Technical Modules

Pipeline Stage	Technical Module / File	Functional Description & Output
Stage 1: Hygiene	src/data_loader.py	Cleans raw Excel, filters returns/fees. Output: 1,036,154 clean records.
Stage 2: Encoding	src/encoder.py	Groups baskets by InvoiceNo into CSR sparse matrix. 99.53% sparsity.
Stage 3: Mining	src/frequent_mining.py	Apriori vs FP-Growth runtime and memory benchmark engine.
Stage 4: Rules	src/rule_generator.py	Extracts rules (Support, Conf, Lift), filters misleading Lift ≈ 1.0 rules.
Stage 5: Insights	src/insights.py src/network_graph.py	Translates rules to business text, runs UK/DE/FR/IE segment mining, builds NetworkX graph.
Stage 6: Scalability	src/scalability.py src/pyspark_fpgrowth.py	Out-of-core 50k-row batching engine & PySpark MLlib distributed architecture.
Stage 7: Web UI	app/streamlit_app.py	Interactive Streamlit dashboard with cross-sell recommendation lookup.

6. Implementation Stack & Tools

Layer	Component / Tool	Rationale & Use Case
Language & Env	Python 3.9, PyYaml	Core execution environment & central configuration file (config.yaml).
Data Processing	Pandas 2.3, SciPy 1.13, NumPy 2.0	Out-of-core DataFrames, CSV/Parquet chunking, & csr_matrix sparse encoding.
Pattern Mining	MLxtend 0.23, PyFPGrowth 1.0	Apriori & FP-Growth itemset mining, association rule extraction.
Graph & Viz	NetworkX 3.2, Matplotlib 3.9, Seaborn 0.13	Product co-occurrence graph layout, EDA histograms & bar charts.
Distributed Scale	Apache Spark / PySpark MLlib	PySpark pyspark.ml.fpm.FPGrowth cluster scaling reference mapping.
Interactive Web UI	Streamlit 1.50	Interactive web app with sidebar controls, sortable tables, & recommender lookup.
Testing & QA	Pytest 8.4	Unit testing framework for data hygiene, encoding, mining, and rule generation.

7. Work Plan & Phase-Wise Milestones

- **Phase 1: Environment Setup & Project Layout (Segment 1)** — Directory structure, virtual environment, dependencies, config.yaml.
- **Phase 2: Data Ingestion & Hygiene Pipeline (Segment 2)** — Cancellation filtering, text standardization, Parquet/CSV output.
- **Phase 3: Transaction Encoding & EDA (Segment 3)** — Sparse matrix creation, basket size stats, EDA visualizations.
- **Phase 4: Frequent Pattern Mining Benchmark (Segment 4)** — Apriori vs. FP-Growth benchmarking across min_supports.
- **Phase 5: Association Rule Generation & Pruning (Segment 5)** — Rule extraction, Lift/Confidence sorting, misleading rule detection.

- **Phase 6: Business Insights & Network Graph Layer (Segment 6)** — NetworkX visualization, regional country analysis (UK, DE, FR, IE).
- **Phase 7: Out-of-Core Scalability & PySpark Architecture (Segment 7)** — 50,000-row chunk processor, StageProfiler, PySpark FPGrowth module.
- **Phase 8: Streamlit Interactive Web Application (Segment 8)** — Web app with parameter sliders, rule explorer, & cross-sell lookup tool.
- **Phase 9: Final Reporting & Synthesis (Segment 9)** — Comprehensive markdown executive report (reports/final_report.md).
- **Phase 10: Testing, Packaging, & Documentation (Segment 10)** — Pytest test suite, main.py CLI runner, README.md setup guide.

8. Expected Deliverables & Outcomes

1. **Cleaned Transactions Dataset:** Processed Parquet & CSV tables (1,036,154 records, 39,517 invoices).
2. **Modular Python Source Code:** Structured codebase in src/ (data_loader, encoder, eda, frequent_mining, rule_generator, network_graph, insights, scalability, pyspark_fpgrowth).
3. **Algorithmic Benchmark Metrics:** Empirical comparison CSV tables and charts detailing Apriori vs. FP-Growth execution runtimes and memory footprints across support values.
4. **Business Strategy & Network Visualizations:** High-resolution product co-occurrence network graphs and regional purchasing segment profiles (reports/insights/).
5. **Interactive Web Dashboard:** Production-ready Streamlit app (app/streamlit_app.py) for live parameter tuning and cross-sell recommendation lookup.
6. **Out-of-Core Engine & PySpark Module:** Scalability module capable of processing 1M+ transaction rows under an 800 MB RAM budget.
7. **Automated Test Suite & CLI Runner:** pytest test suite (tests/test_pipeline.py) passing with 100% success rate and single CLI entry point (main.py).
8. **Final Technical Report & README:** Full documentation in reports/final_report.md and README.md.

9. Risks, Limitations, & Future Extensions

9.1 Risk & Mitigation Plan

- **Risk 1: Combinatorial Memory Explosion at Low Support Values.**
Mitigation: Sparse matrix encoding (csr_matrix), item-support pre-filtering, and max_len=4 constraints prevent out-of-memory errors.
- **Risk 2: Spurious Rules (High Confidence, Low Lift).**
Mitigation: Automated filtering prunes all candidate rules with Lift < 1.2.

9.2 Future Extensions

1. **Multiple Minimum Item Supports (MSApriori):** Assigning custom support thresholds to rare, high-value products to prevent undersampling.
2. **Temporal & Seasonal Association Mining:** Incorporating timestamp seasonality to extract time-varying purchase patterns (e.g., Christmas vs. Summer baskets).
3. **High-Utility Itemset Mining (HUIM):** Factoring item profit margins into itemset scoring to prioritize profit-maximizing bundles over volume-maximizing bundles.

10. References

1. Agrawal, R., & Srikant, R. (1994). *Fast Algorithms for Mining Association Rules in Large Databases*. Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), 487–499.
2. Han, J., Pei, J., & Yin, Y. (2000). *Mining Frequent Patterns without Candidate Generation*. ACM SIGMOD Record, 29(2), 1–12.
3. Chen, D., Sain, S. L., & Guo, K. (2012). *Data mining for the online retail industry: Implicit association rule mining*. Journal of Database Marketing & Customer Strategy Management, 19(3), 197–208.
4. Raschka, S. (2018). *MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack*. Journal of Open Source Software, 3(24), 638.
5. UCI Machine Learning Repository. *Online Retail II Data Set*. Available at: <https://archive.ics.uci.edu/ml/datasets/Online+Retail+II>